

Тема: Свойства ООП и магические методы в разработке приложений.

Теория:

1. Свойства ООП:

Свойство	Описание	Пример из реального приложения
Инкапсуляция	Скрытие внутренних данных и предоставление доступа через методы/свойства	Баланс банковского счёта — приватный атрибут, изменение только через <code>deposit()</code> / <code>withdraw()</code>
Наследование	Создание нового класса на основе существующего	Класс <code>User</code> → <code>Admin</code> , <code>Moderator</code>
Полиморфизм	Единый интерфейс для разных типов данных	Метод <code>render()</code> у классов <code>Button</code> , <code>Checkbox</code> , <code>TextField</code>

2. Магические методы Python (основные для задач):

Метод	Назначение
<code>__init__(self, ...)</code>	Конструктор, инициализация объекта
<code>__str__(self)</code>	Строковое представление для пользователя (<code>print</code>)
<code>__repr__(self)</code>	Строковое представление для разработчика (отладка)
<code>__add__(self, other)</code>	Перегрузка оператора <code>+</code>
<code>__eq__(self, other)</code>	Перегрузка оператора <code>==</code>
<code>__len__(self)</code>	Возвращает длину объекта (функция <code>len()</code>)
<code>__getitem__(self, key)</code>	Доступ по индексу или ключу (<code>obj[key]</code>)
<code>__call__(self, ...)</code>	Позволяет вызывать объект как функцию

Задание.

Задачи на свойства ООП.

Задание 1: Инкапсуляция.

Условие:

Создайте класс `BankAccount` с приватными атрибутами:

- `__account_number` (строка, readonly)
- `__balance` (число, неотрицательное)

Реализуйте:

- Конструктор, принимающий номер счёта и начальный баланс.
- Методы `deposit(amount)` — пополнение.
- Метод `withdraw(amount)` — снятие (если достаточно средств, иначе выбросить исключение).
- Геттер `get_balance()` (без сеттера).
- Свойство `balance` через `@property`.

Вопрос: Какое свойство ООП реализовано?

Задание 2: Наследование.

Условие:

Создайте базовый класс `Vehicle`:

- Атрибуты: `brand`, `model`, `year`
- Метод `info()` — возвращает строку с маркой, моделью и годом.
- Метод `start_engine()` — возвращает "Двигатель запущен".

Создайте два класса-наследника:

- `Car` — добавляет атрибут `number_of_doors` (целое число). Переопределите `info()`, добавив количество дверей.
- `Motorcycle` — добавляет атрибут `has_sidecar` (bool). Переопределите `start_engine()` — возвращает "Рокот мотоцикла".

Вопрос: Какой вид полиморфизма демонстрируется в цикле for?

Задачи на магические методы.

Задание 1: «__str__ и __repr__».

Условие:

Класс `Student` с атрибутами: `name`, `age`, `grade` (средний балл).

Реализуйте:

- `__str__` — выводит "Студент: Иван, возраст: 20, балл: 4.8" (для пользователя).
- `__repr__` — выводит "Student('Иван', 20, 4.8)" (для разработчика).

Задание 2: «__getitem__ и __setitem__».

Условие:

Класс `Library`, который внутри хранит словарь `{название_книги: автор}`.

Реализуйте:

- `__setitem__(self, title, author)` — добавляет/обновляет книгу.
- `__getitem__(self, title)` — возвращает автора или "Книга не найдена".
- `__len__` — возвращает количество книг.
- `__call__(self)` — возвращает список всех названий книг.